Step 1 — The Database and the Safety Guard — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the sample database (seed, schema reader, query runner) and the deterministic `is_safe_select` guard — the parts that hold the real data and stop an unsafe query before it ever runs, with no model call involved at all. **If not:** the guard is the entire safety story of this project; debugging it *and* real model behavior at the same time in Step 3 would be miserable.

Setup files

`requirements.txt`: `langchain-openai`, `openai`, `pydantic`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO` / `DB_PATH=data/shop.db`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `data/*.db`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass

class UnsafeQueryError(Exception):
    """The generated SQL was not a safe, single read-only SELECT."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
```

```
    log_level: str
    db_path: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        db_path=os.getenv("DB_PATH", "data/shop.db"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The sample database

```
# src/db/seed.py
# Build a tiny shop database (only if it isn't there yet): two tables and a
# handful of rows, so questions have real data to run against.
import os
import sqlite3

CUSTOMERS = [
    (1, "Ada", "London"),
    (2, "Grace", "New York"),
    (3, "Alan", "London"),
]

ORDERS = [
    (1, 1, 120.0, "completed"),
    (2, 1, 60.0, "pending"),
    (3, 2, 200.0, "completed"),
    (4, 3, 45.0, "completed"),
    (5, 3, 80.0, "cancelled"),
]

def build_sample_db(db_path):
    if os.path.exists(db_path):
        return
    folder = os.path.dirname(db_path)
    if folder != "":
        os.makedirs(folder, exist_ok=True)
    connection = sqlite3.connect(db_path)
    try:
        connection.execute(
            "CREATE TABLE customers ("
            "id INTEGER PRIMARY KEY, name TEXT, city TEXT)"
        )
        connection.execute(
            "CREATE TABLE orders ("
            "id INTEGER PRIMARY KEY, customer_id INTEGER, "
            "amount REAL, status TEXT)"
        )
        connection.executemany(
            "INSERT INTO customers VALUES (?, ?, ?)", CUSTOMERS
        )
        connection.executemany(
            "INSERT INTO orders VALUES (?, ?, ?, ?)", ORDERS
        )
        connection.commit()
```

```
    finally:
        connection.close()
```

```
# src/db/schema.py
# Return the database's CREATE TABLE statements as one string. The model
# needs the schema to know which tables and columns it can use.
import sqlite3

def schema_text(db_path):
    connection = sqlite3.connect(db_path)
    try:
        cursor = connection.execute(
            "SELECT sql FROM sqlite_master WHERE type = 'table'"
        )
        rows = cursor.fetchall()
    finally:
        connection.close()
    statements = []
    for row in rows:
        if row[0] is not None:
            statements.append(row[0])
    return "\n".join(statements)
```

```
# src/db/runner.py
# Execute a SELECT and return the rows. Only ever called after the guard
# has approved the query.
import sqlite3

def run_select(db_path, sql):
    connection = sqlite3.connect(db_path)
    try:
        cursor = connection.execute(sql)
        rows = cursor.fetchall()
    finally:
        connection.close()
    return rows
```

The safety guard (deterministic, no model)

```
# src/guard.py
# Decide whether a generated query is safe to run. Safe = a single,
# read-only SELECT. Everything else (writes, schema changes, chained
# statements) is rejected. Plain Python, so it tests with no model.
```

```
FORBIDDEN_WORDS = [
    "insert", "update", "delete", "drop", "alter",
    "create", "replace", "truncate", "attach", "pragma",
]

def is_safe_select(sql):
    cleaned = sql.strip().rstrip(";").strip()
    lowered = cleaned.lower()
    if not lowered.startswith("select"):
        return False
    if ";" in cleaned:
        return False
    for word in FORBIDDEN_WORDS:
        if word in lowered:
            return False
    return True
```

The tests (no model)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "SQLSAGE_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")
```

```
if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_guard.py
# Run from the repo root: python -m tests.unit.test_guard
from src.guard import is_safe_select

def test_plain_select_passes():
    sql = "SELECT count(*) FROM orders WHERE status = 'completed'"
    result = "WRONG"
    if is_safe_select(sql):
        result = "OK"
    print(f"case 1 (plain SELECT allowed)     -> {result}")

def test_writes_are_rejected():
    bad = [
        "DELETE FROM orders",
        "UPDATE orders SET amount = 0",
        "INSERT INTO orders VALUES (9, 1, 5, 'x')",
        "DROP TABLE orders",
        "SELECT * FROM orders; DROP TABLE orders",
    ]
    result = "OK"
    for sql in bad:
        if is_safe_select(sql):
            result = "WRONG"
    print(f"case 2 (writes/chained rejected)  -> {result}")

if __name__ == "__main__":
    test_plain_select_passes()
    test_writes_are_rejected()
```

```
# tests/unit/test_db.py
# Run from the repo root: python -m tests.unit.test_db
import os
import tempfile

from src.db.runner import run_select
from src.db.schema import schema_text
from src.db.seed import build_sample_db

def test_seed_and_select():
    folder = tempfile.mkdtemp()
    db_path = os.path.join(folder, "test_shop.db")
```

```
    build_sample_db(db_path)

    rows = run_select(
        db_path, "SELECT count(*) FROM orders WHERE status = 'completed'"
    )
    result = "OK"
    if rows[0][0] != 3:
        result = "WRONG"
    print(f"case 1 (3 completed orders)   -> {result}")

def test_schema_has_tables():
    folder = tempfile.mkdtemp()
    db_path = os.path.join(folder, "test_shop.db")
    build_sample_db(db_path)

    schema = schema_text(db_path)
    result = "OK"
    if "customers" not in schema or "orders" not in schema:
        result = "WRONG"
    print(f"case 2 (schema lists tables)  -> {result}")

if __name__ == "__main__":
    test_seed_and_select()
    test_schema_has_tables()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (plain SELECT allowed)     -> OK
case 2 (writes/chained rejected)  -> OK
```

```
case 1 (3 completed orders)   -> OK
case 2 (schema lists tables)  -> OK
```

The guard and the database are proven with hand-written SQL and a temp file — so Step 3 only has to feed in a model-generated query, not re-test the safety or the execution.